

Lesson 3 — AWS Account, EC2 & Ansible Ad Hoc

Homework · DevOps Course, Cohort 2.3

Topics: creating an AWS account, IAM basics, launching EC2 instances, installing Ansible (control node), building an inventory, ad hoc commands (`ping` , `command` , `shell` , `setup` , `apt` , `service` , `copy` , `file` , `user` , `lineinfile`), idempotency, `--check` / `--diff` .

⚠ **Cost warning:** Stay on Free Tier eligible instance types (`t2.micro` / `t3.micro`). **Terminate your instances when you finish** (see Part E). Leaving instances running is the single most common way people get surprised by a bill.

PART A

Create your AWS account

1. Sign up for a free AWS account (or use one you already have). Use a real email you check — billing alerts land there.
2. Turn on **MFA on the root user** immediately. Never use root day-to-day again after this step.
3. Create a non-root **IAM user** for yourself with console access **and** programmatic access (access key + secret key). Attach a policy that allows EC2 (e.g. `AmazonEC2FullAccess` is fine for a lesson, tighter is better if you already know how).
4. Log out of root, log back in as your IAM user. Everything from here on is done as that user.
5. Set a **budget/billing alert** (Billing → Budgets) for a small amount (e.g. \$5) so you get an email if something is left running by accident.

PART B

Launch VMs in AWS

1. Launch **two** Ubuntu EC2 instances (`t2.micro` or `t3.micro`), same VPC/subnet, named something you'll recognize (`ansible-1` , `ansible-2`). One instance is the minimum; two lets you prove ad hoc commands really run across a fleet, not just one box.
2. Create (or reuse) a key pair, download the `.pem` , and `chmod 400` it.
3. Security group: allow **SSH (22) from your IP only**, and **HTTP (80) from anywhere** — you'll need port 80 later to verify a web server ad hoc.
4. Before touching Ansible, confirm you can connect manually:

```
ssh -i /path/to/key.pem ubuntu@<public-ip>
```

Do this for **both** instances. If this fails, fix it here — Ansible will fail for the exact same reason and won't tell you anything a plain `ssh` wouldn't.

PART C

Install Ansible and build an inventory

1. Install Ansible on your own machine (the **control node** — nothing gets installed on the targets):

```
brew install ansible           # macOS
sudo apt update && sudo apt install -y ansible # Ubuntu/Debian/WSL
python3 -m pip install --user ansible         # fallback, works anywhere
```

Run `ansible --version` and note the `config file`, `python version`, and module search path it prints.

2. Create a working directory and write `inventory.ini` listing both EC2 instances:

```
[web]
ansible-1 ansible_host=<ip-1> ansible_user=ubuntu
ansible_ssh_private_key_file=~/.ssh/key.pem
ansible-2 ansible_host=<ip-2> ansible_user=ubuntu
ansible_ssh_private_key_file=~/.ssh/key.pem

[web:vars]
ansible_python_interpreter=/usr/bin/python3
ansible_ssh_common_args='-o StrictHostKeyChecking=accept-new'
```

3. Write `ansible.cfg` next to it:

```
[defaults]
inventory = ./inventory.ini
host_key_checking = True
```

4. Prove the whole chain works:

```
ansible web --list-hosts
ansible web -m ping
```

Both hosts must reply `pong` before you move on. Paste the output in `answers.md`.

PART D

Ad hoc commands (no playbook yet)

Run each of the following against your `web` group and paste the command **and** its output in `answers.md`.

1. `ansible web -a "uptime"` — implicit `command` module.
2. `ansible web -m shell -a "free -m | grep Mem"` — needs `shell` because of the pipe.
3. `ansible web -m setup -a "filter=ansible_distribution*"` — read off each host's OS from the returned facts.
4. Install `nginx` with the `apt` module, then **run the exact same command a second time** and paste both JSON outputs side by side. Label which field (`changed`) proves idempotency.

```
ansible web -m apt -a "name=nginx state=present update_cache=yes" -b
```

5. Make sure the service is running and enabled:

```
ansible web -m service -a "name=nginx state=started enabled=yes" -b
```

6. Push a file and confirm it's live — the ad hoc way, not with a browser:

```
echo "<h1>Hello from Ansible</h1>" > index.html
ansible web -m copy -a "src=index.html dest=/var/www/html/index.html mode=0644" -b
ansible web -a "curl -s localhost"
```

7. Create a `deploy` user with sudo rights on both hosts:

```
ansible web -m user -a "name=deploy shell=/bin/bash groups=sudo append=yes" -b
```

8. Harden SSH config with `lineinfile`, **dry run first**:

```
ansible web -m lineinfile -a "path=/etc/ssh/sshd_config regexp='^PermitRootLogin'
line='PermitRootLogin no'" -b --check --diff
```

Explain in `answers.md` what `--check --diff` told you would change, then run it for real (without `--check`) and confirm.

9. Target only one host with `--limit` (e.g. `ansible web --limit ansible-1 -m ping`).
10. Deliberately run `ansible doesnotexist -m ping` against a group name that doesn't exist and paste the error. Explain in one line how this differs from an `UNREACHABLE!` error.

PART E

Tear down (required)

Terminate both EC2 instances, delete the security group you created, and confirm in the console that nothing is left running. Note in `answers.md` why a stopped-but-not-terminated instance (or an unattached Elastic IP) can still cost money.

Deliverables

- `answers.md` — every command from Parts C and D with its pasted output, plus the written explanations asked for in Parts A, D.8, D.10, and E.
- Screenshots (or console output) of: the IAM user you created, the two running EC2 instances, the `nginx` page reachable via public IP, and the final terminated/torn-down state.

Bonus

- Add a second inventory group, e.g. `[db]`, containing just one of your two hosts, and show `ansible db -m ping` working alongside `ansible web -m ping`.
- In 5–8 lines: why is Ansible called "agentless," and what specifically does that save you compared to an agent-based tool like Puppet or Chef?
- Do Part D entirely with `-f 10` (raise the fork count) and explain what forks control.